
FluxReel

AMD GPU-Powered Short-Video Creation Tool

Project Profile Document

Track 1 — Multimodal Content Creation Tools

AMD AI DevMaster Hackathon · 2026-07

	Zhiwei Li	Thomas Yang (Aztice)
Role	Senior AI Full-Stack Engineer	HK secondary school student · AI infra engineer
Experience	12 years engineering & entrepreneurship	Multiple years in AI infrastructure
GitHub	github.com/lzwjava	github.com/aztice

One topic · one AMD Radeon GPU · one 15-second video —fully local.

1 Project Background

Short video has become the dominant content format across platforms such as Douyin, WeChat Video Account, and Reels. Yet producing an explainer video today is a manual, multi-step chore: write a script, source or generate images, add captions, pick music, and edit everything together — typically **hours of work per video**. Existing automation options are either cloud APIs that *charge per video* or tools that *upload content to the cloud*, which is a deal-breaker for privacy-sensitive material.

FluxReel solves this with a single pipeline: one plain-text topic in, one polished **15-second vertical short video** (1080×1920, 9:16, 30 fps) out — with all core multimodal inference running **fully local on an AMD Radeon GPU** using the ROCm software stack.

- **No per-video API cost** — image generation runs on the local Radeon GPU, not a paid cloud endpoint.
- **Privacy by design** — scripts, images, captions and the final MP4 are produced on the user’s own machine.
- **Bilingual output** — the topic language is auto-detected (English / 中文) and captions are rendered with CJK-aware fonts and character-based wrapping.
- **Turnkey tooling** — a CLI, a web UI and a REST API, plus optional YouTube upload.

FluxReel is built for **Track 1 — Multimodal Content Creation Tools** of the AMD AI DevMaster Hackathon, and is implemented as the *fluxreel* Python package (source: <https://github.com/lzwjava/amd-hackathon-lzwjava>).

2 Target Users & Application Scenarios

FluxReel targets creators who need **fast, high-volume, cost-controlled** short-video output:

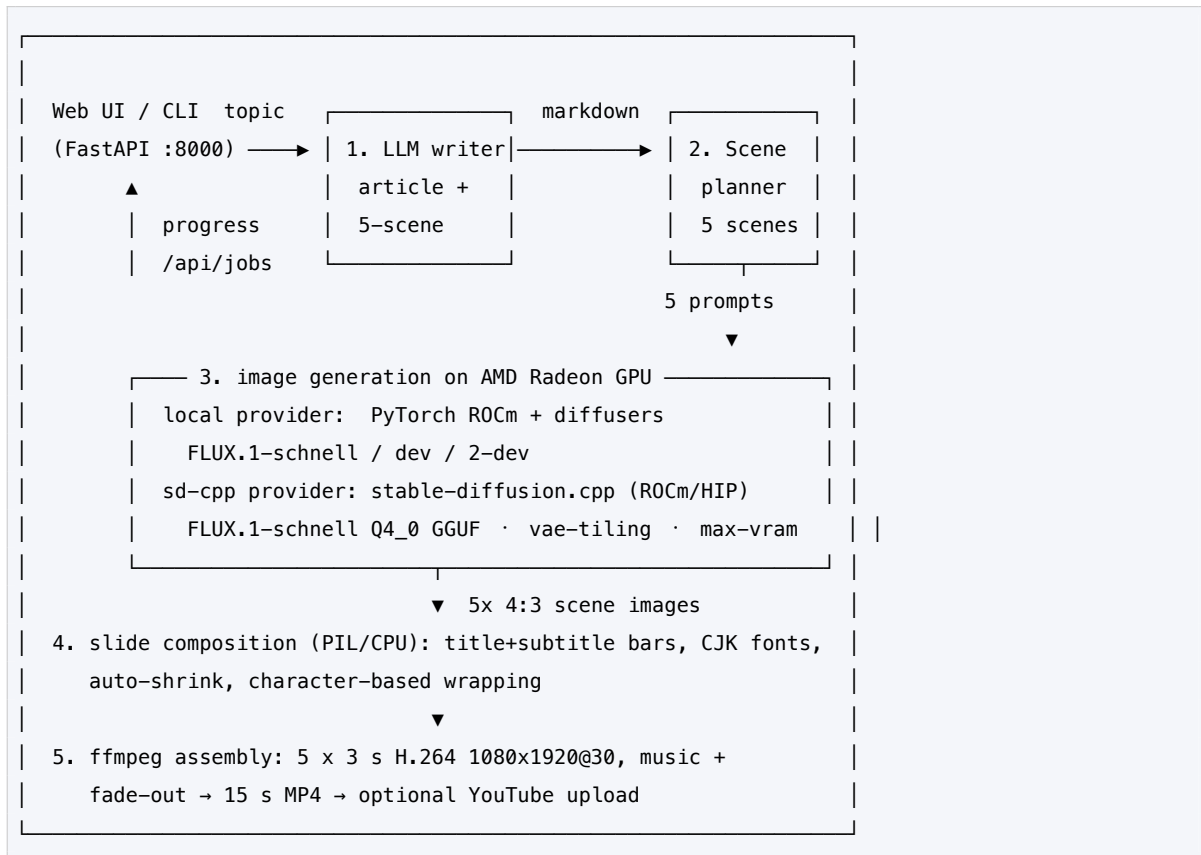
Scenario	What the user gets
New-media operations	A complete explainer video from a one-line topic — draft, scenes, images, captions, music — in minutes instead of hours . Douyin / WeChat Video Account / Reels ready.
Personal creation	Custom styles (cyberpunk, infographic, dark neon) with zero design skills required.
Commercial visual design	Consistent brand-style scene images and captions for product explainers and marketing clips.
Education / knowledge sharing	Students and teachers produce an explainer video for any topic (GPUs, RISC-V, AI concepts, ...) with bilingual (EN/中文) caption support.

- **Input:** a one-line topic, e.g. “How GPUs work” or “GPU 是什么?”.

- **Output:** a 15-second, 1080×1920 @ 30 fps MP4 with 5 scenes, title/subtitle bars, background music and fade-out.
- **Deployment:** local machine with an AMD Radeon GPU (12 GB-class VRAM), or a rented AMD GPU box; the CLI also ships remote-server management (tunnels, model downloads, GPU info).

3 System Architecture

FluxReel is a five-stage pipeline from topic to MP4, orchestrated by a FastAPI server with an embedded web UI and an asynchronous job queue (progress polling, MP4 download, optional YouTube upload).



3.1 Provider abstraction

Image generation is pluggable behind a common interface (**ImageProvider**): a factory creates one of four providers per job (`--provider`):

3.2 Inference placement

4 Model & Algorithm Introduction

4.1 Models

Provider	Engine	Use case
local	PyTorch ROCm + diffusers, FLUX.1-schnell / dev / 2-dev	Default AMD GPU path, full-fidelity
sdcpp	stable-diffusion.cpp, FLUX.1-schnell Q4_0 GGUF	Quantized fast path, low VRAM
openrouter	Cloud FLUX API (optional)	Convenience only; never required for the core
auto	local first, fallback to cloud	Robustness on unknown machines

Stage	Where it runs
Scene image generation (core)	AMD Radeon GPU via ROCm — PyTorch/diffusers or stable-diffusion.cpp (HIP)
Slide composition, caption layout	Local CPU (PIL)
ffmpeg assembly	Local CPU (H.264, no re-encode)
Script / scene planning (LLM)	Optional: local LLM on the same Radeon GPU (vLLM / llama.cpp); a cloud LLM is only a convenience, never required for the image/video core

4.2 Generation algorithm

FLUX.1-schnell is a 12B-parameter rectified-flow transformer that denoises Gaussian latents into images conditioned on CLIP-L and T5-XXL text encodings. Its **guidance-distilled** variant (schnell) produces high-quality images in only **4 Euler steps** with **cfg-scale 1.0** (no classifier-free guidance), which avoids the 2× batch overhead of CFG entirely.

- **Quantization.** The Q4_0 GGUF path quantizes the diffusion transformer to **4-bit block-wise weights** (via stable-diffusion.cpp), cutting the working set from tens of GB to **~8.75 GB per run** on a 12 GB-class card.
- **VAE decoding.** The 16-channel latent decoder runs in **tiles** (`--vae-tiling`), so large images decode without OOM on small VRAM.
- **Text encoding.** CLIP-L and T5-XXL run on **CPU** while the diffusion transformer and VAE run on the **GPU**, trading a little latency for much lower VRAM pressure.

4.3 Caption / typography algorithm

- Large dark title bars (110 px) and subtitle bars (64 px) sized for Douyin/WeChat-safe layouts; **fonts auto-shrink** to fit the bar width.
- **Character-based wrapping** (CJK-aware, Noto Sans CJK) — Chinese has no spaces, so wrapping must be per-character; this is handled correctly for 中文 as well as English.

Model	Params	Role	Why
FLUX.1-schnell	12B	scene images (primary)	4-step distilled — 7× faster than 28-step
FLUX.1-dev / FLUX.2-dev	12B	scene images (quality)	28 steps when fidelity matters
FLUX.1-schnell Q4_0 GGUF	12B→ 4-bit	scene images (quantized)	4× smaller; fast GPU/CPU mix
LLM (optional)	7–70B	script + scene planning	Text only; vLLM / llama.cpp on ROCm or API

5 Adaptation for AMD Radeon GPU / ROCm

5.1 Software stack

1. **ROCm runtime** — `amdgpu-install` (driver + runtime) or the ROCm container on the AMD GPU host.
2. **PyTorch for ROCm** — `pip install torch` from download.pytorch.org/whl/rocmX.Y, plus `diffusers transformers accelerate`.
3. **stable-diffusion.cpp** — compiled with a **HIP** backend; the `sd-cli` binary drives the quantized Q4_0 GGUF path (`SDCPP_BIN` / `SDCPP_MODEL_DIR`).
4. **Verification** — `fluxreel info` reports GPU, ROCm, PyTorch build, VRAM and disk.
5. **Optional local LLM** — `vllm serve` or `llama.cpp` on ROCm for fully-offline script planning.

5.2 The sd-cpp quantized fast path

The `sdcpp` provider spawns a **fresh sd-cli process per image** (no persistent pipeline, no VRAM leaks) and serializes runs with a mutex so that the 5 parallel scene threads never contend for the GPU. A typical invocation is:

```
sd-cli --diffusion-model flux1-schnell-Q4_0.gguf \
  --vae ae.safetensors --clip_l clip_l.safetensors \
  --t5xxl t5xxl_fp16.safetensors --prompt "<prompt>" \
  --cfg-scale 1.0 --sampling-method euler --steps 4 \
  --width 960 --height 720 --seed 42 --vae-tiling \
  --max-vram 10 --output scene_000.png \
  --backend diffusion=cuda,clip=cpu,vae=cuda,t5xxl=cpu
```

5.3 Optimizations on the Radeon GPU

Optimization	Mechanism	Effect
Distilled inference	FLUX.1-schnell, 4 steps vs. 28	7× fewer denoising steps per image
Q4_0 quantization	4-bit block-wise GGUF	4× smaller weights; ~8.75 GB per run on 12 GB cards
VAE tiling	<code>--vae-tiling</code>	Tile-based decode, no OOM on small VRAM
VRAM budget	<code>--max-vram 10</code>	Headroom for desktop use; stable on 12 GB-class cards
Backend offload	<code>diffusion=cuda, vae=cuda clip/t5xxl=cpu</code>	Text encoders off-GPU, lower VRAM pressure
No CFG	<code>--cfg-scale 1.0</code>	Saves 2× batch compute (distilled model)
Parallel scenes	ThreadPoolExecutor + GPU lock	Total latency $\approx$ 1 image + scheduling, not 5
Early encoding	Per-slide H.264, then concat	No full-video re-encode; fast, deterministic
Resolution discipline	4:3 960×720 matched to slide area	Fewer pixels, less GPU time per image
Optional ROCm LLM	vLLM / llama.cpp on the same GPU	Fully offline script planning

5.4 Measured characteristics

- **Scene image:** 2–15 s depending on variant (4-step schnell $\approx$ seconds; 28-step dev slower) at 960×720–1024×768 on a single Radeon GPU.
- **Full 15 s video:** ffmpeg assembly is CPU-bound, 10–30 s, independent of GPU step count.
- **Memory:** FLUX.1-schnell Q4_0 GGUF fits comfortably in 12 GB-class VRAM with `--max-vram 10` + VAE tiling (8.75 GB per run, leaving 2.9 GB for desktop use).